INTERVIEW STUDY GUIDE

AssignmentHub — full-stack interview prep

Every question a reviewer is likely to ask about this project — the OnnoRokom Projukti Assistant Software Engineer recruitment build — with answers grounded in the actual codebase, not textbook generalities. Organized so you can jump straight to a section, search across everything, or read it start to finish.

---

11 sections **75** questions **53/53** backend tests passing **6/6** frontend tests passing

.NET 8 · **Next.js 16**

01 Project & Requirements

What the brief actually asked for, and the calls made where it left things open.

What is this project, in one sentence?

A role-based Assignment & Submission Management System for a school or college — Admin, Teacher, and Student each get a distinct workflow around creating, publishing, submitting, and grading assignments — built as the take-home project for OnnoRokom Projukti Limited's Assistant Software Engineer recruitment.

What exactly does each role do?

- **Admin** — manages users, classes, subjects, teacher-to-subject assignments, and generic app settings; has read-only oversight of every assignment and submission platform-wide.
- **Teacher** — creates/updates/deletes/publishes assignments for classes and subjects they're specifically assigned to; grades submissions with marks and feedback; can return a submission for correction.
- **Student** — sees only *published* assignments for classes they're enrolled in; submits and can update an answer before the deadline; sees their own marks and feedback once graded.

What were the mandatory technical requirements from the brief?

Frontend: Next.js/React/TypeScript, responsive UI, form validation, API integration. Backend: ASP.NET Core Web API in C#, REST, validation, error handling, logging, Swagger. Database: PostgreSQL or MongoDB with proper relationships. Auth: JWT plus role-based authorization. Testing: unit tests covering important business rules, authorization, and the submission workflow.

What was explicitly optional, and which of those did you still build?

The brief called a live URL, Swagger URL, Docker, pagination, advanced filtering, and notifications all optional. I built Docker as the *primary* setup path rather than a nice-to-have, pagination and search/sort on every list endpoint, and real-time SignalR notifications — then went further with rate limiting, soft deletes, a grading audit trail, health checks, and request correlation IDs, since none of those needed anything the brief didn't already ask me to justify architecturally.

Why PostgreSQL over MongoDB?

The domain is strongly relational — Users, Classes, Subjects, Assignments, and Submissions are tied together by several many-to-many and one-to-many relationships, and there are hard invariants (like "exactly one submission per student per assignment") that map directly onto a relational unique constraint. MongoDB would mean either duplicating relational data across documents or doing manual multi-document joins in application code, for no benefit — there's no need here for schema flexibility or horizontal document scale.

What assumptions did you make where the brief left things open, and where are they documented?

- Submissions are text/Markdown only, no file upload — keeps the evaluation environment dependency-free.
- No public self-registration; only an Admin creates accounts, and the very first Admin exists solely via the startup seed routine.
- "Change submission status when necessary" is a distinct endpoint from grading, not folded silently into it, since the brief lists them as separate Teacher responsibilities.
- Application settings are a generic key/value store rather than a fixed settings schema.

All of it is written down in the README's "Design Decisions & Assumptions" and "Known Limitations" sections — the brief explicitly asks for assumptions to be documented, not just made silently.

02 Architecture & Design Decisions

Clean Architecture on the backend, feature-layered on the frontend — and how both are mechanically enforced, not just documented.

Why Clean Architecture for a project this size — isn't that overkill?

The brief is explicitly evaluating "understanding of requirements, system design" — architecture is part of what's graded, not incidental to working code. It also directly serves the brief's own requirements: resource-based authorization and deadline logic are pure business rules that live in Domain/Application and can be unit-tested with zero database, which is exactly what "unit tests covering business rules" is asking for. And the dependency rule — Api → Infrastructure → Application → Domain, never backward — is mechanically enforced by NetArchTest, so it can't silently rot the way an unenforced convention would.

Walk me through the four backend layers.

- **Domain** — entities, enums, domain exceptions. Zero project references, no persistence or framework knowledge, holds real behavior (`Assignment.Publish()`, `Submission.Grade()`).
- **Application** — DTOs, repository interfaces, service interfaces + implementations, FluentValidation validators. Depends only on Domain; defines interfaces Infrastructure implements (Dependency Inversion); forbidden from referencing any `Microsoft.AspNetCore.*` namespace.
- **Infrastructure** — EF Core DbContext/configurations/migrations, repository implementations, JWT/password hashing, SignalR, the resource-ownership authorization handler. Implements Application's interfaces.
- **Api** — controllers, middleware, `Program.cs` composition root. Thin: calls Application services only, never touches the DbContext directly.

How do you actually enforce the architecture instead of just documenting it?

`NetArchTest.Rules` in a dedicated ArchitectureTests project, run as ordinary xUnit facts in CI: Domain has no dependency on Application/Infrastructure/Api; Application has no dependency on Infrastructure/Api and none on any `Microsoft.AspNetCore` namespace; the `AssignmentHub.Api.Controllers` namespace has no dependency on `AssignmentHub.Infrastructure.Persistence`. A violation fails `dotnet test` exactly like a broken business-rule test would — it's a build gate, not a lint suggestion.

What's the frontend equivalent of that layering?

A feature-layered structure: `app/` (routing and composition only), `features/<name>/` (vertical slices — api, hooks, schema, types, components), `shared/` (design-system primitives, hooks, the axios client, cross-cutting types) — with the same inward-only dependency rule, enforced by `eslint-plugin-boundaries`: `app` can import `features` and `shared`; a feature can import `shared` and its own files; `shared` can import nothing from either. It's a lint *error*, so `npm run lint` — which CI runs — fails on a violation.

You allowed two exceptions to that frontend rule — what and why?

`features/academics` may import `features/users` (to populate Teacher/Student picker dropdowns for teacher-assignment and enrollment forms), and `features/assignments` may import `features/academics` (for the Class/Subject picker when creating an assignment). Both are one-directional and explicitly named in the ESLint policy rather than left as an unenforced convention — duplicating another feature's list-fetch logic per-consumer would have been the worse trade.

Doesn't Application depending on `Microsoft.EntityFrameworkCore` break "framework-agnostic"?

There's a difference between a persistence *abstraction* package and a hosting *framework*. EF Core's `IQueryable` async extensions (`FirstOrDefaultAsync`, `CountAsync`, …) back the generic `PaginatedList<T>.CreateAsync` helper and let services compose LINQ over a repository's `Query()`. Application never references Npgsql or a concrete `DbContext`. The architecture test checks "no dependency on `AssignmentHub.Infrastructure`" and "no dependency on `Microsoft.AspNetCore`" — not "zero NuGet packages" — which is the same pragmatic line drawn by mainstream Clean Architecture templates.

Why not MediatR / CQRS?

Nothing in the plan called for command/query separation, and adding MediatR would introduce commands, handlers, and pipeline behaviors without a requirement driving it. A thin controller calling one Application service method is simpler and equally testable — CQRS here would be exactly the kind of premature abstraction worth avoiding.

03 Backend — ASP.NET Core & C#

Controllers, middleware, error handling, logging — the plumbing that keeps business logic isolated.

Why .NET 8 specifically?

It's the current Long-Term-Support release, the safest choice for something meant to be evaluated and potentially maintained afterward. It also ships the built-in rate-limiting middleware and the Node.js-runtime hosting maturity this build relies on.

How do you keep controllers thin?

A global MVC filter, `ValidationFilter`, runs before every action: it reflects over the action's arguments, resolves a matching FluentValidation `IValidator<T>` for each, and short-circuits with a 400 ProblemDetails on failure — so controllers never call a validator directly. Beyond an ownership check where one applies, every action does exactly one thing: call a single Application service method and translate the result to an HTTP status.

How does global error handling work?

A single `ExceptionHandlingMiddleware` wraps the whole pipeline and maps exception types to RFC 7807 ProblemDetails: `KeyNotFoundException` →404, `UnauthorizedAccessException` →401, the domain's `ForbiddenOperationException` →403, `DeadlinePassedException` / `DuplicateSubmissionException` / `DbUpdateConcurrencyException` →409, `InvalidMarksException` /FluentValidation's `ValidationException` →400, everything else→500 with a generic message while the real exception is logged server-side via Serilog and never leaked to the client.

Why specific Domain exceptions instead of error codes returned from services?

It keeps the "what HTTP status does this mean" decision in exactly one place — the middleware — instead of scattered through every controller. A service method just states what happened (`DeadlinePassedException`, `DuplicateSubmissionException`), which doubles as documentation of the business rule and is exactly what gets unit-tested directly.

Explain the DTO strategy.

Every bounded context owns its `Dtos/` folder — response DTOs are immutable C# records, request DTOs mirror the FluentValidation rules for that action. There's no shared "God DTO"; `AssignmentDto`, `SubmissionDto`, etc. each carry only what that screen needs, including denormalized display fields (`className`, `subjectName`, `teacherName`) so the frontend never needs a second round trip just to render a row.

Why static mapping methods instead of AutoMapper?

Three reasons: AutoMapper's recent major versions introduced commercial licensing terms for larger organizations — worth flagging rather than adopting silently; reflection-based mapping hides bugs until runtime and is harder to step through than a five-line method; and at this entity count, a `ToDto()` extension method is barely more code than configuring an AutoMapper profile, while staying fully debuggable with zero extra dependency risk.

Why Serilog, and what does the correlation ID actually buy you?

Serilog gives structured logging — each entry is named properties, not just an interpolated string — with sinks to console and a rolling daily file. A `CorrelationIdMiddleware` stamps every request with an ID (reusing an inbound `X-Correlation-Id` header if present) and pushes it into Serilog's `LogContext`, so every log line for one request — across middleware, controller, service, repository — can be grepped by that single ID. That's the difference between "something failed somewhere" and actually tracing one user's failing request through the whole call stack.

What does the health check endpoint actually verify?

`/health` uses `AddDbContextCheck<AppDbContext>()`, which runs a lightweight real query against the database — so it verifies actual connectivity, not just "the process didn't crash." It's wired into the Docker Compose backend service's own container `HEALTHCHECK` via `curl`, which I installed explicitly in the runtime image since the base ASP.NET image doesn't ship it.

Why .NET 8's built-in rate limiter instead of a NuGet package?

`Microsoft.AspNetCore.RateLimiting` has shipped in the shared framework since .NET 7 — no extra dependency, no extra config format. A fixed-window limiter (10 requests/minute per client IP) is applied only to `/api/auth/login` and `/api/auth/refresh` via `[EnableRateLimiting("auth")]`, blunting brute-force attempts without pulling in a Redis/memory-store dependency a third-party package would want.

04 Database & EF Core

The schema, the constraints that do real work, and one provider-specific bug that only showed up under test.

Walk me through the core entities and relationships.

`User` (Admin/Teacher/Student via a `Role` enum) has many `RefreshToken`s. `Class` and `Subject` are joined by `ClassSubject`, which also carries the assigned `TeacherId` — unique on `(ClassId, SubjectId)`, one teacher per class/subject at a time. `StudentClass` joins Student↔Class as an enrollment fact (composite key, no surrogate Id). `Assignment` belongs to one `ClassSubject` and one `CreatedByTeacher`. `Submission` belongs to one `Assignment` and one `Student`, unique on `(AssignmentId, StudentId)`. `GradeAuditLog` and `Notification` are append-only side records; `Setting` is a generic key/value row.

Why a unique index on `(AssignmentId, StudentId)`?

It's the database-level enforcement of "exactly one submission per student per assignment." The Application layer also checks for an existing submission before inserting and throws a specific `DuplicateSubmissionException` for a clean 409 — but the constraint is the fact of record that survives even a bug or a race condition the service code doesn't anticipate. Defense in depth, not either/or.

How do soft deletes work, and why global query filters instead of checking a flag everywhere?

Entities that should never truly disappear implement an `ISoftDelete` marker interface. In `AppDbContext.OnModelCreating`, a small reflection loop finds every entity implementing it and applies `HasQueryFilter(e ⇒ !e.IsDeleted)` automatically. Every repository's `Query()` / `GetByIdAsync` is transparently filtered — no service method anywhere has to remember to add that check, which removes an entire class of "forgot to filter deleted rows" bugs.

What's the "xmin" concurrency token, and what did it break?

PostgreSQL has no native `ROWVERSION` column; every Postgres table has a hidden system column, `xmin`, that changes on every update, which EF Core can treat as an optimistic-concurrency token via a shadow property — the correct idiomatic choice against real Postgres. But when integration tests later ran against SQLite in-memory (to keep CI fast and dependency-free), SQLite has no concept of a real `xmin` system column, so `EnsureCreatedAsync()` generated an ordinary NOT-NULL column nothing populated, and every insert failed. The fix: move that one piece of configuration out of the shared entity configuration and into `AppDbContext.OnModelCreating`, gated behind `if (Database.IsNpgsql())` — so it only ever applies against the real provider.

Why Code-First migrations instead of a hand-written schema?

The brief specifically requires the evaluator to set up the database "without manually creating tables." `Program.cs` calls `dbContext.Database.MigrateAsync()` on every startup, so `docker compose up` alone creates the full schema. Code-First also means the C# entity model is the single source of truth — schema drift is caught at migration-generation time, not discovered in production.

How is pagination implemented generically?

A shared `PaginationParams` (Page, PageSize, Search, SortBy, SortDir) binds automatically from the query string, and a generic `PaginatedList<T>.CreateAsync(IQueryable<T>, page, pageSize)` helper runs `CountAsync` then `Skip` / `Take` / `ToListAsync` against whatever queryable a service composed — every list endpoint gets the same `?page=&pageSize=&search=&sortBy=&sortDir=` contract without duplicating counting logic per feature.

05 Authentication & Authorization

Tokens, ownership checks, and the tradeoffs made explicit rather than hidden.

Walk me through login end to end.

The client POSTs email/password to `/api/auth/login`. `AuthService` looks up the user, verifies the password via ASP.NET Core Identity's `PasswordHasher<T>` (PBKDF2 — never a custom hash), and on success generates a short-lived (15-minute) JWT access token plus a long, cryptographically random refresh token. The refresh token is hashed with SHA-256 — deliberately not PBKDF2, since it's already 256 bits of entropy, not a low-entropy human password, so a slow hash would only add latency — and stored in `RefreshTokens`. Both tokens and the user's role/name come back in the response; the frontend stores them and attaches the access token as a Bearer header on every later request.

What happens when the access token expires?

The axios response interceptor catches a 401 and — unless the failing request was itself login/refresh — calls `/auth/refresh` with the stored refresh token, deduplicating concurrent refresh attempts behind one shared promise. The backend validates the token isn't expired/revoked, revokes it, and issues a brand-new pair (rotation on every use, so a stolen refresh token has a very short usable window). The original request is retried once with the new access token; if refresh itself fails, the session is cleared and the user is sent to `/login`.

Role-based vs resource-based authorization — why did you need both?

`[Authorize(Roles = "Teacher")]` answers "is this caller a Teacher at all" — a coarse gate. It can't answer "does this teacher own this specific assignment," which is exactly the brief's requirement. For that I built a custom `AuthorizationHandler<AssignmentOwnershipRequirement, Guid>`: the controller calls `authorizationService.AuthorizeAsync(User, assignmentId, new AssignmentOwnershipRequirement())` before update/delete/publish/view-submissions, the handler asks Application's `IsOwnedByTeacherAsync( ... )` (or auto-succeeds for Admins), and returns `Forbid()` on failure. A plain role check literally cannot express that rule — it needs the specific resource in hand.

Why does the authorization handler live in Infrastructure, not Application?

`Microsoft.AspNetCore.Authorization` is a framework-hosting concern, and Application needs to stay 100% framework-agnostic (verified by the NetArchTest rule). So the actual business logic — "does teacher X own assignment Y" — is a plain method in Application, while the ASP.NET-Core-specific plumbing (`IAuthorizationHandler` / `IAuthorizationRequirement`) lives in Infrastructure, which already has a framework reference for SignalR and Identity. It's a tighter boundary than my own first sketch of the plan drew.

Why aren't the frontend's auth cookies httpOnly? Isn't that a gap?

It's a documented tradeoff, not an oversight. A true httpOnly refresh-token cookie can't be read by client JS to attach as an Authorization header, which would require proxying every API call through a Next.js Route Handler — a full Backend-for-Frontend layer, real added complexity for this project's scope. Instead the app accepts the same XSS-exposure most SPAs already accept with a localStorage-held JWT, and mitigates it with a short access-token lifetime and refresh-token rotation. It's called out explicitly in the README rather than silently claimed as fully secure.

Why store the role in a separate cookie instead of decoding the JWT client-side?

Next.js route guarding (the Proxy) runs at the edge/server layer and needs a synchronous, cheap answer to "is this authenticated, and what role" with no network call or JWT-parsing on every request. A plain `ah_role` cookie set at login is a direct read via `request.cookies.get( ... )`. It's an optimistic check only — the backend remains the sole authoritative enforcement — which matches Next.js's own documented guidance for Proxy-level auth checks.

There's no public sign-up. How does the very first Admin account get created?

The idempotent seed routine (`DatabaseSeeder.SeedAsync`, guarded by `if (await context.Users.AnyAsync()) return;`) creates the first Admin directly at startup. Every Teacher/Student account after that is created by an Admin through `POST /api/users` — matching the brief's role model, which has no public registration at all.

06 Frontend — Next.js & React

App Router specifics, data fetching, forms, and the real-time notification bell.

What actually changed in Next.js 16 that affected this build?

Next.js 16 renamed the `middleware.ts` file convention to `proxy.ts` — same runtime behavior, new file and export name (`middleware.js` is now deprecated). I found this by reading the framework's own bundled docs before writing the route-guarding code, since a stale mental model would have shipped a deprecated convention on day one. Dynamic route `params` / `searchParams` are also Promises that must be awaited in Server Components, and a Client Component reading a dynamic route's params uses React's `use()` hook to unwrap that promise synchronously.

Why App Router over Pages Router?

Pages Router is effectively legacy at this point. App Router gives Server Components by default (less client JS for pages that don't need interactivity), colocated layouts per route segment — used here for the three role shells — and is the framework's actively developed direction.

Why TanStack Query instead of `useEffect` + fetch, or Redux?

Every screen here is "fetch a list/detail, mutate, reflect the result" — exactly TanStack Query's use case. It gives request deduplication, cache keyed by a query key, automatic refetch-on-mutation-success via `invalidateQueries`, and loading/error state without hand-rolling any of it. Redux would add a global store and reducers for what is, in every case here, server state rather than client-only UI state.

Explain the axios interceptor and token-refresh flow.

A single configured axios instance attaches the Bearer token from cookies on every request via a request interceptor. The response interceptor maps any error's ProblemDetails body into a typed `ApiError`, and specifically on a 401 that isn't itself a login/refresh call, deduplicates concurrent refresh attempts behind one shared promise, retries the original request once with the new token, and on ultimate failure clears the session and hard-navigates to `/login`.

Why React Hook Form + Zod over Formik or plain controlled state?

Zod gives one schema that's both the TypeScript type source (`z.infer<typeof schema>`) and the runtime validator, so a form's type and its validation rules can never drift apart. React Hook Form is uncontrolled by default — fields don't re-render the whole form on every keystroke — and wires to Zod with a single `resolver: zodResolver(schema)`, noticeably less boilerplate than Formik for the same guarantee.

You hit a real TypeScript issue with `z.coerce.number()` — what was it?

`z.coerce.number()` has an *input* type of `unknown` and an *output* type of `number` — zodResolver's inferred generic didn't line up with `useForm<T>`'s single type parameter, so TypeScript rejected the resolver. Rather than split `z.input`/`z.output` and fight React Hook Form's generics, I used a plain `z.number()` in the schema and let RHF coerce at the DOM boundary via `register("marksAwarded", { valueAsNumber: true })` — the schema only ever sees a real number.

How does real-time notification work on the frontend?

A single SignalR `HubConnection` connects to `/hubs/notifications`, authenticated by passing the JWT as a query-string parameter via `accessTokenFactory` — browsers can't set an Authorization header on a WebSocket upgrade, so the backend's JWT handler has a dedicated hook that only accepts that query-string token for paths under `/hubs`. A `useNotificationSocket()` hook, mounted once per role layout, listens for `AssignmentPublished`/`SubmissionGraded` events and calls `queryClient.invalidateQueries(["notifications"])` — the backend always writes the Notification row before pushing, so the refetch is guaranteed to already see it.

What's the actual difference between `useRoleGuard` and the Proxy — isn't one redundant?

They cover different windows. The Proxy runs server-side on the initial navigation/RSC request and is the primary guard. `useRoleGuard`, called inside each role layout's Client Component, is a second check for a client-side transition rendering a layout without a fresh server round trip. Neither is the source of truth either way — the backend API enforces the real rule regardless of what either frontend guard does; both are UX-layer defense-in-depth only.

How is the visual design related to the provided `education-theme` template?

That static template only ships marketing/auth pages — `index`, `login`, `register`, `contact` — with zero dashboard, table, or grading layouts. It was used strictly as a visual-language reference: its accent colors (a blue/teal Material palette) were sampled and refined into Tailwind's `@theme` tokens, and the login page borrows its layout register. Every dashboard, table, and grading screen was built from scratch, and that distinction is stated explicitly in the README rather than implied.

07 Testing Strategy

53 backend tests, 6 frontend tests, and the two real bugs that surfaced while wiring them up.

Give me the exact test breakdown.

36 unit tests — Domain-entity logic with zero mocks (`Assignment.Publish` / `AcceptsSubmissions` / `IsPastDeadline` boundaries, `Submission.Grade` boundary and exception cases), Identity (JWT claims/expiry, password-hash round trip), an authorization-handler test (Admin bypass, Teacher-owns, Teacher-doesn't-own), and Application-service tests via Moq. **7 architecture tests** — the NetArchTest layering rules. **10 integration tests** — real HTTP requests through the real ASP.NET Core pipeline, real auth, real authorization, against a SQLite in-memory database. All 53 pass; the frontend adds 6 Jest tests on top.

Moq can't fake EF Core's async LINQ provider — how did you unit-test a method that calls `Query().Include().FirstOrDefaultAsync()`?

A plain `list.AsQueryable()` throws at runtime the moment code calls `FirstOrDefaultAsync` / `AnyAsync` / `CountAsync` on it, because LINQ-to-Objects' provider doesn't implement `IAsyncQueryProvider`. `MockQueryable.Moq` is built specifically for this — it wraps an in-memory collection in a queryable whose provider does support those async extensions, so `mockRepo.Setup(r ⇒ r.Query()).Returns(data.BuildMock())` behaves like a real EF-backed queryable for `.Include()` / `.Where()` / `.FirstOrDefaultAsync()` chains.

Why SQLite in-memory for integration tests instead of a real Postgres test container?

It keeps `dotnet test` fast, hermetic, and runnable in CI with zero external services — no Testcontainers or Docker-in-Docker needed. `CustomWebApplicationFactory` swaps `AppDbContext`'s registration to a single open SQLite `:memory:` connection kept alive for the whole test collection, builds the schema via `EnsureCreatedAsync()` — not the committed migrations, which contain Npgsql-specific column types SQLite can't parse — then seeds via the exact same `DatabaseSeeder.SeedAsync` used in production, so the tests exercise the real seed path rather than a bespoke fixture.

You mentioned a `WebApplicationFactory` config-timing bug — walk me through it.

`Program.cs` reads `Jwt:Secret` synchronously inside `AddJwtAuthentication`, before `builder.Build()` runs. `WebApplicationFactory`'s `ConfigureAppConfiguration` overrides, though, aren't actually applied until *during* that same `Build()` call — so a test-only JWT secret supplied that way was invisible to the code that had already baked the token-validation key from the checked-in (empty) `appsettings.json` value. The result: login *issued* tokens signed with the test secret while the bearer handler *validated* against the empty one — every authenticated test request failed with a signature-mismatch 401. Fixed with `builder.UseSetting(key, value)` instead, which seeds configuration early enough in host construction that both the startup-time read and every later live read see the same value.

How did you unit-test the frontend's route-guarding logic, given it's a Next.js Proxy function?

`NextRequest` / `NextResponse` are real, constructable classes exported from `next/server`, so the test imports the actual exported `proxy` function and calls it directly with hand-built requests — setting a raw `cookie` header the way a browser would — and asserts on the returned response's redirect `Location` header or its absence. No mocking framework, no running Next server; it's a pure-function test against the real code path.

What did you choose to unit test on the frontend, and why those two things specifically?

The deadline-gating logic in `SubmissionForm` — disabled/locked once the deadline has passed with no still-editable submission, read-only once Graded, editable otherwise — because it's the frontend mirror of the single most safety-critical rule in the app. And the Proxy's route-guarding decisions, because that backs up the brief's "role-based access enforced by the backend" requirement on the client side too.

08 DevOps — Docker & CI/CD

One command to a running stack, and the build-context bug that nearly broke it.

Walk me through what `docker compose up --build` actually does.

Three services: `postgres` (with a `pg_isready` healthcheck), `backend` (`depends_on` postgres with `condition: service_healthy`, so it never starts before the database can accept connections), and `frontend` (`depends_on` backend for build ordering; the browser calls the backend directly, not container-to-container). On startup `Program.cs` unconditionally calls `MigrateAsync()` then `DatabaseSeeder.SeedAsync` — skipped only under the "Testing" environment integration tests use — so one command creates the schema and seeds demo data with no manual step, exactly satisfying the brief.

You hit a real Docker build bug — what was it?

Neither project had a `.dockerignore`, so `COPY . .` copied my local `bin/` / `obj/` folders into the build context — including MSBuild's cached `project.assets.json` and `.nuget.g.props/.targets` files, which embed absolute Windows paths (a Visual Studio NuGet fallback folder like `C:\Program Files (x86)\...`). Inside the Linux build container, `dotnet publish --no-restore` tried to reuse that poisoned cache and failed immediately with a "can't find fallback package folder" error. Adding `.dockerignore` files excluding `bin/`, `obj/`, `node_modules/`, `.next/` forced a clean, container-native restore and fixed it.

Why multi-stage Dockerfiles, and what's in each stage?

Backend: an SDK-image stage for restore/publish, then a much smaller ASP.NET runtime image that only copies the published output — plus `curl` installed explicitly so the container's own `HEALTHCHECK` can hit `/health`. Frontend: a `deps` stage (`npm ci`, cached separately so a source change doesn't force a reinstall), a `build` stage (`NEXT_PUBLIC_API_BASE_URL` passed as a build `ARG`, since Next.js bakes `NEXT_PUBLIC_*` vars into the client bundle at build time, not read at container start), and a runtime stage that copies only `next.config.ts`'s `output: "standalone"` artifact — a self-contained `server.js` with the minimal `node_modules` subset it actually needs.

What does each CI workflow actually check?

`ci-backend.yml`: restore, build Release, then `dotnet test` across all three test projects — including the architecture-layering rules and the SQLite-backed integration tests — with zero external services required. `ci-frontend.yml`: `npm ci`, `npm run lint` (which includes the ESLint architecture-boundary rule, so a stray cross-feature import fails CI, not just a review comment), `tsc --noEmit`, `npm test`, then `npm run build`. Both are path-filtered so a change to one stack doesn't trigger the other's pipeline.

If this had to run in real production instead of a recruiter's laptop, what changes first?

Real secrets management instead of a `.env` file for the JWT secret and DB password; HTTPS termination (currently plain HTTP, intentionally, for local/demo simplicity); the httpOnly-cookie/BFF-proxy pattern traded away for scope; a managed Postgres instance with real backups instead of a named Docker volume; and probably `IMemoryCache` or distributed caching on the read-heavy Classes/Subjects endpoints, which was explicitly scoped out as non-load-bearing at this dataset size.

09 Business Logic Deep Dives

Full request walk-throughs for the rules the brief cares about most.

Walk me through a student submitting an assignment, end to end.

`POST /api/student/assignments/{id}/submit` → the controller calls `SubmissionService.SubmitAsync(assignmentId, studentId, request)`. The service loads the Assignment, checks `assignment.AcceptsSubmissions(utcNow)` — must be Published and `utcNow ≤ DeadlineUtc` — throwing `DeadlinePassedException` or a generic "not open" error if not; checks no submission already exists for `(assignmentId, studentId)`, throwing `DuplicateSubmissionException` if it does; otherwise creates a new `Submission` (Status = Pending), saves, and returns the fully-loaded DTO. Every branch maps to the right HTTP status via the global middleware — the controller itself never inspects the outcome.

Walk me through publishing an assignment, including the notification.

`PUT /assignments/{id}/publish` → the controller first runs the resource-ownership check (`Forbid()` if this teacher doesn't own it) → `AssignmentService.PublishAsync` loads the assignment with its ClassSubject included, calls the domain method `assignment.Publish()` (Draft→Published), saves, looks up every student enrolled in that class, writes one `Notification` row per student, saves again, then calls `IRealtimeNotifier.NotifyAssignmentPublishedAsync` to push over SignalR. The database write always happens before the push, so a client that refetches notifications right after receiving one is guaranteed to already see it.

How exactly is a Teacher stopped from grading another Teacher's submission?

Unlike Assignment CRUD (which uses the imperative `IAuthorizationService` handler at the controller boundary), grading's ownership check happens *inside* `SubmissionService.GradeAsync` via `IsGradableByTeacherAsync` — it checks whether the submission's Assignment has `CreatedByTeacherId == teacherId` OR `ClassSubject.TeacherId == teacherId`, throwing `ForbiddenOperationException` before touching anything if neither holds. Since grading only ever has one code path — there's no separate "grade" resource the way there's a separate Assignment resource — embedding the check in the service was simpler and equally safe.

Why is "Returned" a separate status from "Pending," and what triggers it?

The brief lists "change the submission status when necessary" as a distinct Teacher responsibility from "assign marks and provide feedback," so it has its own endpoint (`PUT /submissions/{id}/status`) and domain method (`Submission.ReturnToStudent(feedback)`) rather than being folded into grading. A teacher can return a Pending or already-Graded submission to reopen it; a student can resubmit to any submission that's Pending or Returned (`CanBeModifiedByStudent` — only Graded locks it), and resubmission flips the status back to Pending for re-review.

How do you guarantee a Draft assignment is invisible to students — not just hidden in the UI?

`AssignmentService.GetPublishedForStudentAsync` filters `Status == Published` directly in the database query — a Draft row is never fetched, let alone serialized, so there's no field for a client to accidentally leak. An integration test logs in as a real seeded student and asserts the returned list never contains the seeded Draft assignment's title, over a real HTTP call through the real pipeline — not a mock that could pass while the actual endpoint is broken.

How are deadlines kept from being a client-trust problem?

Every `DeadlineUtc` is stored and compared in UTC exclusively on the server. `Assignment.IsPastDeadline(DateTime utcNow)` / `AcceptsSubmissions(utcNow)` take "now" as a parameter, supplied by an injected `IDateTimeProvider` rather than calling `DateTime.UtcNow` directly — specifically so unit tests can freeze time at an exact boundary. The frontend's `shared/lib/date.ts` is the only place a UTC value is ever converted to the viewer's local time, purely for display; the submission/update endpoints re-check the deadline server-side regardless of what a disabled button on the client claims.

10 Behavioral — Prove You Understand It

The questions meant to test whether you actually own this codebase.

What's the hardest bug you ran into, and what did it teach you?

The Postgres `xmin` concurrency token breaking SQLite-backed integration tests. It's a reminder that a provider-specific optimization needs a provider-specific guard the moment more than one provider touches the same EF model — the fix was gating the configuration behind `Database.IsNpgsql()` rather than either ripping out a legitimate feature or hardcoding a workaround into shared entity configuration.

If you had another week, what would you add or change?

The httpOnly-cookie/BFF-proxy auth pattern traded away for scope; `IMemoryCache` on the read-heavy Classes/Subjects endpoints; file/attachment upload for submissions (explicitly out of scope per the stated assumption); and probably giving the Submissions controller's grading action its own authorization-handler-based check for symmetry with Assignments, even though the in-service check today is equally correct.

How confident are you the authorization rules actually work, versus "the code looks right"?

The authorization integration test logs in as two different real seeded teachers, has Teacher 1 create/list an assignment, then switches to Teacher 2's token and attempts a `PUT` on Teacher 1's assignment — asserting a real 403 through the real HTTP pipeline, real JWT validation, and the real `AssignmentOwnershipHandler`. That's a materially stronger claim than a unit test mocking the handler's dependencies, because it can't pass if the actual DI wiring or middleware order is wrong.

What's the single most "senior" decision in this codebase?

Separating the resource-ownership *check* (a plain, framework-agnostic boolean method in Application) from the resource-ownership *enforcement mechanism* (an ASP.NET Core `IAuthorizationHandler` in Infrastructure). It's a small thing, but it's exactly the boundary that keeps a "Clean Architecture" claim from being decorative. The same instinct shows up in choosing plain `z.number()` over fighting Zod's coercion generics, and in gating the `xmin` token to the real provider instead of papering over a symptom.

What would you do differently about scope, in hindsight?

Caching was explicitly cut, and that was the right call for this dataset size — but it's worth being ready to explain *why* rather than just that it's missing: adding `IMemoryCache` to a read-heavy endpoint with no invalidation story is worse than no caching at all, and doing it properly (invalidate on every admin write to Classes/Subjects) was more scope than the brief's "optional extras" list justified alongside SignalR, rate limiting, and the audit trail already delivered.

11 Core Concept Refreshers

Fast, precise definitions for the terms this project leans on — in case you're asked to explain one cold.

What is REST, in a sentence you'd say out loud?

An architectural style for HTTP APIs where each URL identifies a resource (`/assignments/{id}`) and the HTTP verb (GET/POST/PUT/DELETE) states the action, so the API's shape mirrors the domain's nouns rather than exposing remote-procedure-call-style endpoints.

What is a JWT, and what are its three parts?

A JSON Web Token — a compact, signed token with three base64url-encoded parts separated by dots: a header (algorithm/type), a payload (claims — here: user id, email, role, name, expiry), and a signature (HMAC-SHA256 over the first two parts using a server-held secret) that lets the server verify the token wasn't tampered with, without needing a database lookup on every request.

What is Dependency Injection, and why does ASP.NET Core build it in?

A class declares what it needs via constructor parameters (interfaces, not concrete types) rather than constructing its own dependencies — a container wires the real implementations in at runtime. It's what lets `AssignmentService` depend on `IAssignmentRepository` without knowing or caring that it's backed by EF Core and Postgres, which is the entire mechanism Clean Architecture's Dependency Inversion relies on.

`IQueryable<T>` vs `IEnumerable<T>` — what's the actual difference?

`IEnumerable<T>` represents an in-memory sequence — any `.Where()` / `.OrderBy()` chained onto it runs in the application's own memory. `IQueryable<T>` builds an expression tree that the underlying provider (EF Core, here) translates into SQL and executes in the database — so `repository.Query().Where(a ⇒ a.Status == Published)` becomes a `WHERE` clause in Postgres, not a full table scan pulled into .NET first.

What does `async` / `await` actually buy you here?

While a request is waiting on I/O — a database query, an HTTP call — the thread isn't blocked; it's returned to the pool to serve other requests, and resumes this one when the I/O completes. For a web API handling many concurrent requests against a database, that's the difference between needing one thread per in-flight request and comfortably serving far more concurrent traffic on the same thread pool.

What is CORS, and why did this project need to configure it?

Browsers block a page from one origin (`localhost:3000`) from calling an API on a different origin (`localhost:5000`) unless the API explicitly allows it via `Access-Control-Allow-Origin` response headers. Since the frontend and backend are genuinely separate origins here (and separate containers in Docker), the backend configures an explicit CORS policy naming the frontend's exact origin — never a wildcard — with credentials allowed, since the frontend sends cookies.

Optimistic vs pessimistic concurrency — which does this app use, and where?

Pessimistic concurrency locks a row for the duration of an operation, blocking other writers. Optimistic concurrency lets everyone proceed and only checks for conflict at save time — which is what the `Assignment` entity's Postgres `xmin` token does: if two requests load the same assignment and both try to save, the second save's row-version check fails and EF Core throws a concurrency exception (mapped to a 409), rather than either write silently overwriting the other.

What's the N+1 query problem, and how does this codebase avoid it?

Loading a list of parents, then lazily loading each one's related child data in a separate query per row, turns "1 query" into "1 + N queries." This app avoids it by eagerly composing `.Include()` / `.ThenInclude()` directly in the service's query (e.g. `AssignmentService`'s shared `IncludeAll()` helper pulls ClassSubject, Class, Subject, and CreatedByTeacher in the same query) before materializing — one round trip per list, regardless of row count.

What's an ORM, and what's the actual tradeoff of using one?

An Object-Relational Mapper (EF Core, here) lets you work with C# objects and LINQ instead of hand-written SQL, translating queries and tracking changes for you. The tradeoff is control: an ORM can generate suboptimal SQL for a complex query, and hides exactly the kind of provider-specific behavior (like the `xmin`/SQLite mismatch) that only surfaces when you look closely — which is why understanding what SQL your LINQ actually produces still matters.
